A Cost-Effective Model for Fall Prediction Among Adults

Full Name: Chao Tan, Matrikelnummer: xxxxxx
Friedrich-Alexander-Universität Erlangen-Nürnberg
Machine Learning and Data Analytics Lab
https://www.mad.tf.fau.de/
Carl-Thiersch-Straße 2b, 91052 Erlangen, Deutschland
Email: name@fau.de

***Abstract*—Falls cause 684,000 deaths globally each year, making them the second leading cause of accidental death. When people fall, resulting in extreme injuries but without timely treatment, people tend to suffer secondary damage. To mitigate this issue, accurate motion analysis and predictive models are essential for early alert, especially for elderly people, which facilitates prompt medical intervention. This study utilized the public "SisFall" dataset, Which contains data collected by two different types of three-axis accelerometers and a gyroscope to record participants' activities, such as standing, sitting, walking, running, fall, etc. This study employed machine learning algorithms: Random forest (RF), Support Vector Machine (SVM), XGBoost, Artificial Neural Network (ANN) for fall prediction. Our goal is to extract the minimal number of features and find a cost-effective model to predict fall with respect to a set of carefully designed metrics. After comparing the the Statistical Measures (sensitivity(SE), specificity (SP) and accuracy (AC)), CPU usage (CU), and processing time (PT), the results indicate that Random Forest (RF) is the optimal algorithm with the highest score of 34.327. Our contribution is the method to identify the features correlated to the prediction and how to calculate the score representing the total performance of each model.**

I. Introduction

Public health status serves as a critical metric for measuring the extent of society advancement and personal development. As the global population ages, it is inevitable to face the risk of fall and the subsequent injuries. In 2021, approximately 45.6 million falls were reported among individuals aged 65 and older [3]. The increasing problems of fall cases together with the psychological issues pose a significant burden on modern healthcare systems.

In the paper [2], the authors aimed to create a dataset, extract and discuss several features to predict fall. They used different types of accelerometers and one gyroscope to measure activities of the participants and extracted 14 features from the raw data. By applying different equations, these features were able to distinguish fall from activities of daily living (ADL). Their procedures included a preprocess, feature extraction, classification, and validation.

Here are the features [2] extracted for prediction. One sample of acceleration in the three axis is defined as the $\vec{a} = [a_x, a_y, a_z]^T \in R^3$, the sliding window used for computing the dynamic features is denoted with $\tilde{a}[k] = [\vec{a}^T[k - N_v + 1], ..., \vec{a}^T[k]]^T \in R^{Nv \times 3}$, at time-sample k, where N_v is the number of samples in the selected window. The standard deviation operator is defined as $\sigma(\cdot)$, and RMS refers to the Root Mean Square value. The integrals were computed with the trapezoid method, with limits k - N_v + 1 to k. The most suitable features for fall detection were C_2, C_3, C_8, C_9, $and\, C_{13}$. The equations are illustrated in Table I [2]:

Table I: Analysis of test size on algorithms[4]

$Code$	Equation and Feature
C_2	$C_2[k] = \sqrt{a_x^2[k] + a_z^2[k]}$ Sum vector magnitude on horizontal plane
C_3	$C_3[k] = RMS(max(\tilde{a}[k]) - min(\tilde{a}[k]))$ Maximum peak-to-peak acceleration amplitude
C_8	$C_8[k] = \sqrt{\sigma_x^2[k] + \sigma_z^2[k]}$; with $\sigma_i = std(\tilde{a}_i[k])$ Standard deviation magnitude on horizontal plane
C_9	$C_9[k] = \sqrt{\sigma_x^2[k] + \sigma_y^2[k] + \sigma_z^2[k]}$ Standard deviation magnitude
C_{13}	$C_{13}[k] = \int \sqrt{\tilde{a}_x^2[n] + \tilde{a}_z^2[n]} dn$ Activity signal magnitude area on horizontal plane

The authors only compared how these features affected the accuracy for young and elderly participants [2]. They concentrated on providing the freely available dataset. Another paper [4] compared different machine learning algorithms but only focused on the accuracy with a part of the original samples, which contained six young adults of code "SA02, SA05, SA10, SA15, SA20, SA23" and two elders of code "SE06, SE10". The result is shown in Table II,

Table II: Analysis of test size on algorithms[4]

Accuracy		
Test size	Random Forest	Support vector machine
0.20	99.2%	99.3%
0.25	99.2%	99.22%
0.30	99.3%	99.18%
0.35	99.1%	99.25%
0,40	99.2%	99.22%

We aimed to find a cost-effective algorithm for fall prediction with optimal statistical measures (SM), low CPU usage (CU) and short processing time (PT). To achieve this goal, we only considered traditional machine learning algorithms since deep learning algorithms have inherent disadvantages when dealing with simple classification tasks

in restricted devices. Another reason was that we aimed to adapt to outdated computer with lower energy consumption, so that we extended the lifespan of old hardware, saved energy and therefore alleviated the burden to environment. This study utilized the public fall dataset "SisFall" [2], which contained multiple types of activities performed by young and elderly participants. In order to maintain sample integrity and represent as large rang of people in daily life as possible, we included all participants. Furthermore, in order to reduce noise and redundancy, we implemented the same preprocess as the authors did [2]. Afterwards we manually selected the most important features and examined them by using t-SNE tool and RF model. T-SNE is an useful tool to visualize the features in lower dimensions. We also validated the reduction results to ensure it did not compromise the Statistical Measures (sensitivity (SE), specificity (SP) and accuracy (AC)). Finally the low-dimensional data were input into these models: Random forest (RF), Support Vector Machine (SVM), XGBoost, Artificial Neural Network (ANN). We compared these metrics [2] :
i) Statistical Measures as SM. (SE, SP and AC)
ii) CPU usage (CU)
iii) Processing time(PT)
These three Statistical Measures are formulated in Equation (1)[5],

$$SE = \frac{TP}{TP+FN}, SP = \frac{TN}{TN+FP}, AC = \frac{SE+SP}{2} \quad (1)$$

where TP and TN are the true positives and true negatives; FP and FN the false positives and false negatives, respectively [5].

II. Methods

The process pipeline is shown in Fig. 1 with a brief explanation. We aimed to find a cost-effective solution to predict fall with optimal statistical measurements, low CPU usage and short processing time. After extraction, we had 99 statistical features with redundancies and correlations. If we combine all possible features and iteratively compute SM, then we would have 6.33×10^{29} types of combinations to compare, making the solution impossible to converge. To address it, we applied a Heatmap diagram to show correlation between each pair of features. Then we reduced nearly three quarters of the features according to the Heatmap diagram and validated the reduction with RF. Later we manually selected and deleted the remaining features one by one and used t-SNE tool to visualize the distinguishability, which is an evident advantage to select features comparing to the paper [2] and [4].

A. Data acquisition and preprocessing

The data for this study were collected from the SisFall dataset recorded with two accelerometers and one gyroscope, which were all fixed to the waist of the participants and recorded for different test length. The acceleration data acquired from ADXL345 sensor were used in this work, as it is energy efficient and provides the larger span. We followed the same procedures to retain acceleration data from

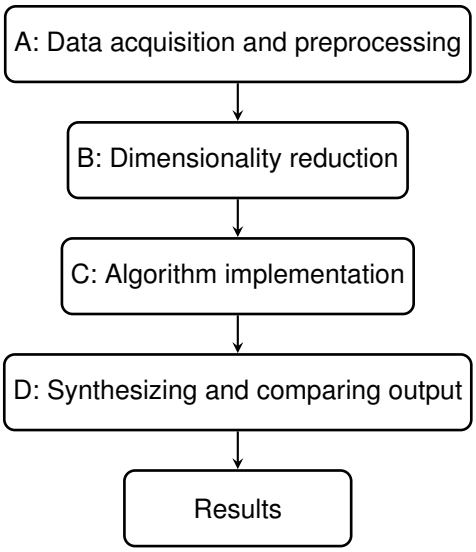


Fig. 1: Process pipeline

ADXL345 and ignored MMA8451Q. In this preprocessing, we concatenated those records of sensory output files and applied 4th order IIR (infinite impulse response) Butterworth low-pass filter with cut-off frequency 5 Hz. We down-sampled the filtered data by half and segmented the data in a time-window within 3 seconds around the peak values of the magnitude in each channel, which formed the basis of the data features. Subsequently, we computed the minimum, maximum, mean, median, range, standard deviation, kurtosis and skewness. For each channel. Consequently we had 99 features, which contained a large amount of redundant information, therefore led to high memory consumption and increased computational overhead. To address these problems, dimensionality reduction was applied as described in Section B.

B. Dimensionality reduction

We examined the distinguishability of these 99 features by inputting the data into t-SNE model (t-Distributed Stochastic Neighbor Embedding). In case that t-SNE shows positive results and omit negative ones, we deliberately input a 3-feature-dataset (simulating extremely inferior reduction only with min_Ax, max_Ax, mean_Ax） into the t-SNE and RF models. T-SNE exhibits the indistinguishability in Fig. 3.

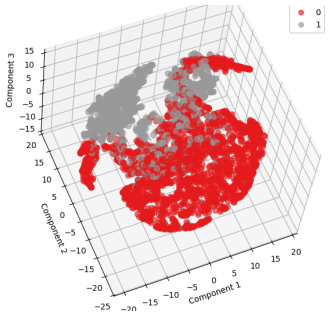


Fig. 3: Extreme dimensionality reduction

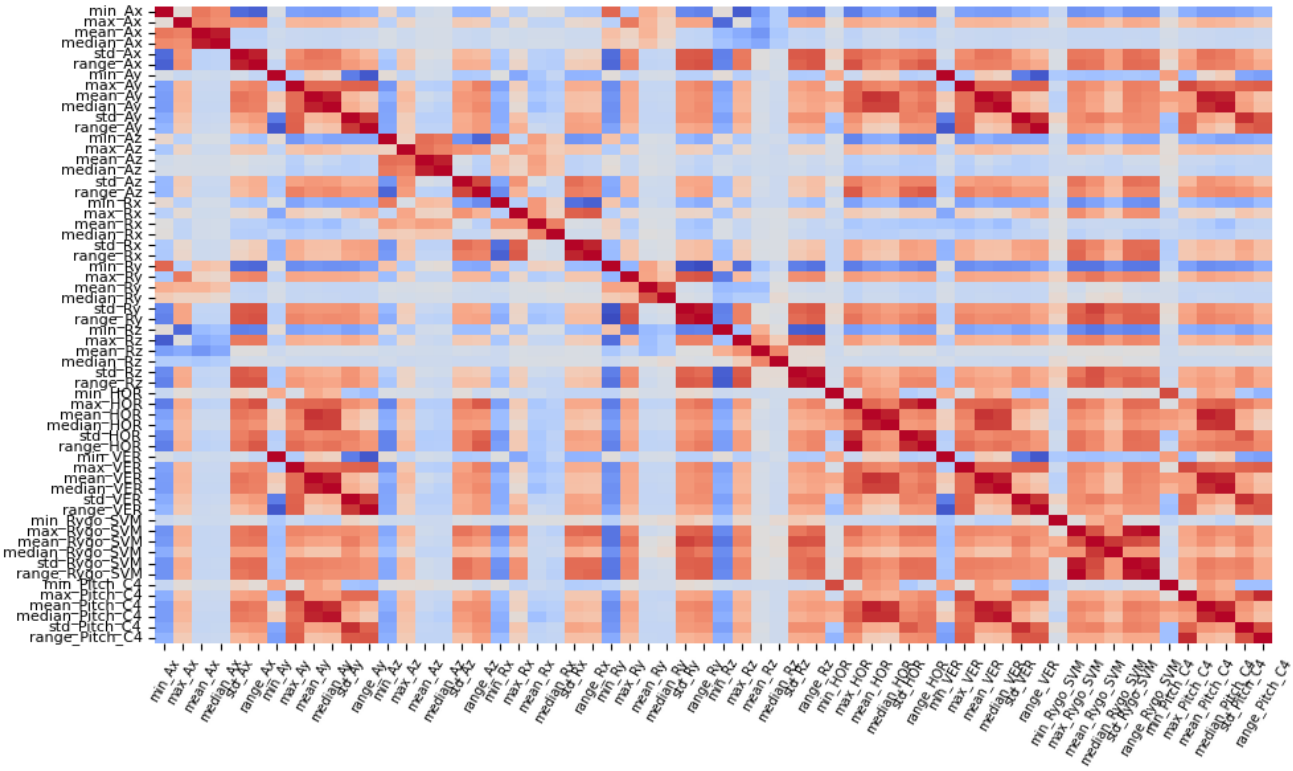


Fig. 2: Heatmap showing the feature correlation

Table III: Poor predictions precision

Metrics	Before dim. reduction	After dim. reduction
AC	0.9931 ± 0.0096	0.9164 ± 0.0089
SE	0.9887 ± 0.0219	0.8403 ± 0.0440
SP	0.9960 ± 0.0108	0.9651 ± 0.0179

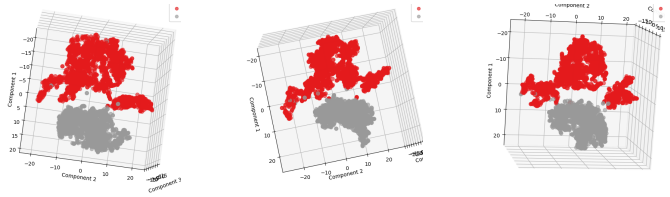
In Table III, RF models also confirms this poor SM: all metrics drop far more than 0.7%, which we set as the criterion for feature selections. If one of the three metrics (AC, SE, SP) dropped more than it, we would stop dimensionality reduction.

After confirming the distinguishability of the dataset and the functionality of t-SNE tool, we computed the correlation of each features and showed the result in a heatmap. The correlation factor is compute by Equation (2), and show in Fig. 2.

$$r_{xy} = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2}\sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}} \tag{2}$$

In Equation (2), r_{xy} denotes the correlation between features x and y, representing different color depth in Fig. 2. A darker-colored area implies a stronger correlation between these two feature and vice versa.

As is shown in this figure, the first quarter of the features in the vertical axis are correlated to the last half features in the horizontal axis. Hence we chose the first 25 features and gradually reduced the dimensionality. This constant processes together with examinating results were shown in Fig. 4.



(a) 25 features (b) 18 features (c) 12 features

Fig. 4: T-SNE showing distinguishability

We also input the 12-feature-dataset into RF models to make prediction and compared the accuracies, which were shown in Table IV.

Table IV: Prediction precision with 12 features

Metrics	Before dim. reduction	After dim. reduction
AC	0.9931 ± 0.0096	0.9920 ± 0.0053
SE	0.9887 ± 0.0219	0.9929 ± 0.0127
SP	0.9960 ± 0.0108	0.9910 ± 0.0194

The accuracy only dropped slightly less than 0.11% and our acceptant drop is less than 0.7%.

C. Algorithm implementation

We followed the following four rules to achieve fairness when measuring:
1. To prevent the different distribution between the training dataset and the test dataset affecting SM, we employed 10-fold cross validation, ensuring each subset had equal probability of being used in the test dataset.
2. We started the timer before reading the preprocessed dataset and stopped it after finishing prediction and outputting the results. This ensured a fair measurement of the elapsed time.
3. We applied those algorithms in an old laptop with a freshly re-installed Windows 10 system so as to eliminate interference of other software.
4. The performance directly connected to SM were computed with equations (1).we had 3 metrics related to SM: sensitivity (SE), specificity (SP) and accuracy (AC). In addition, we also needed to consider CU and PT. Ideally, a robust algorithm should achieve high accuracy while maintaining low CPU usage and short processing time.We multiplied CU by PT, inverted the products and normalized them to acquire NECF (Normalized energy consumption factor). Then we added them to SM metrics and computed Exp(•) of the summation. By this method all metrics in a same scale were synthesized into a score, representing the general performance with an enough large gap between different algorithms.
5. We also compared the SM of each algorithm before and after dimensionality reduction to further validate the feature selection mechanism. 6. The best algorithm with balanced performances “won the game”.

Because we already knew the results of RF, we continued to compute the other three algorithms before and after dimensionality reduction, which can be seen in Fig. 5, 6, 7.

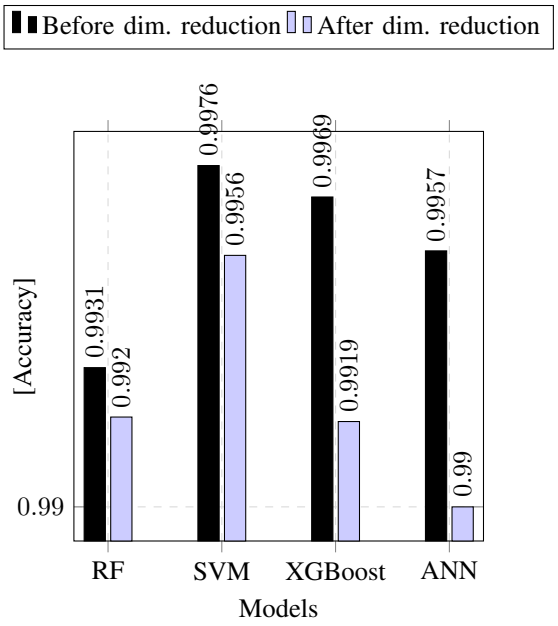


Fig. 5: Accuracy between different models

In Fig. 5, the largest dropping takes place in the accuracies of ANN, i.e. 0.57% and all achieve more than 99% in accuracy.

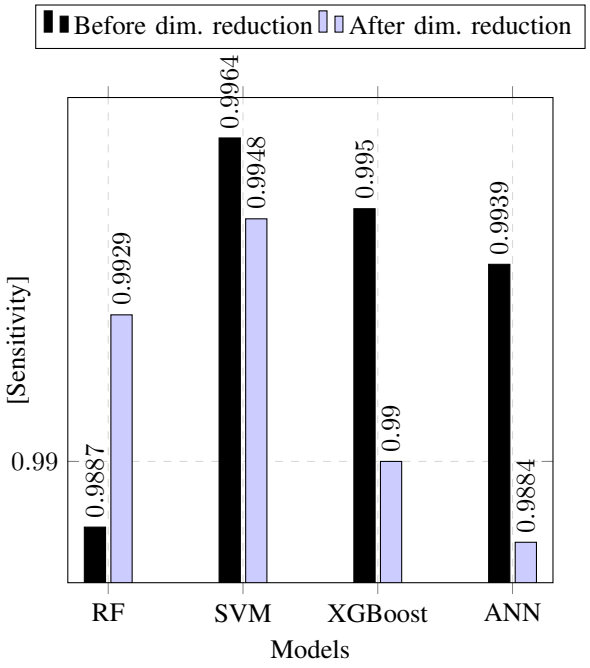


Fig. 6: Sensitivity between different models

In Fig. 6, the largest dropping takes place in the accuracies of ANN, i.e. 0.55% and all achieves more than roughly 98% in Sensitivity.

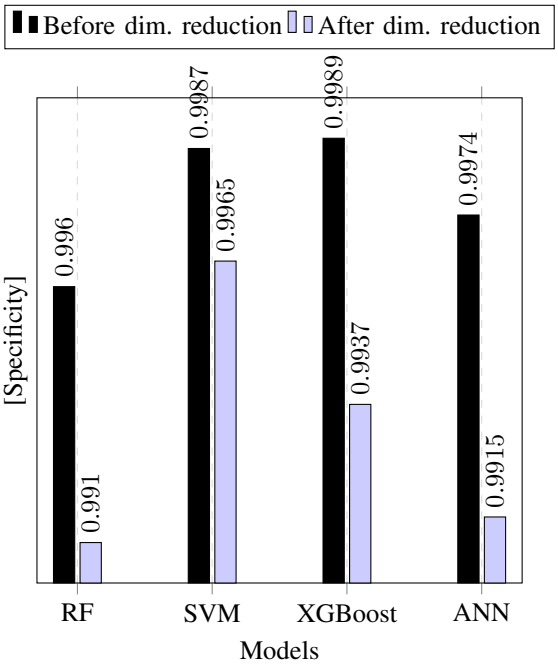


Fig. 7: Specificity between different models

In Fig. 7, the largest dropping takes place in the accuracies of ANN, i.e. 0.59% and all achieves more than 99% in Specificity.

From aforementioned SM, all metrics of these algorithms after dimensionality reduction, drops less than 0.7%, which indicates our dimensionality reductions did not compromise SM. We introduced the CU and PT metrics to measure the economy of different models, which are shown in Fig. 8, 9.

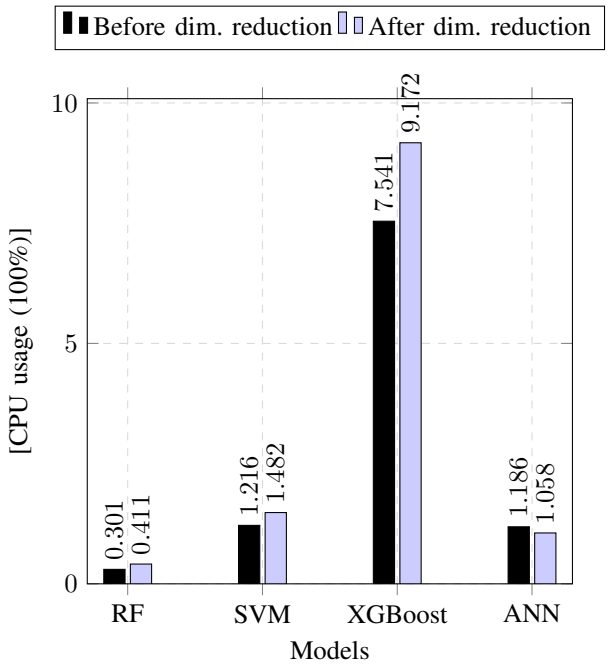


Fig. 8: CPU usage between different models

in Fig. 8, the largest increase of the CU takes place in XGBoost, i.e. 163.1% (9.172-7.541), which is also the highest CU of all models. Meanwhile, RF model has the lowest CU. In Fig. 9, ANN model has used the most time, while XGBoost the least.

D. Synthesizing and comparing output

To evaluate the overall performance of each algorithm, we multiplied CU by PT, inverted the products and normalized them to acquire NECF. Then we added them to SM metrics and computed Exp(•) of the summation. Each step is shown in Table V and VI.

Table V: Metric summary for each algorithm

Models	PT(s)	CU	AC	SE	SP
RF	6.12	**0.411**	0.992	0.9929	0.9910
SVM	4.71	1.482	0.9956	0.9948	0.9965
XGBoost	0.74	**9.172**	0.9919	0.9900	0.9937
ANN	**43.36**	1.058	0.9900	0.9884	0.9915

$$R_{model} = \frac{1}{PT_{model} \times CU_{model}} \tag{3}$$

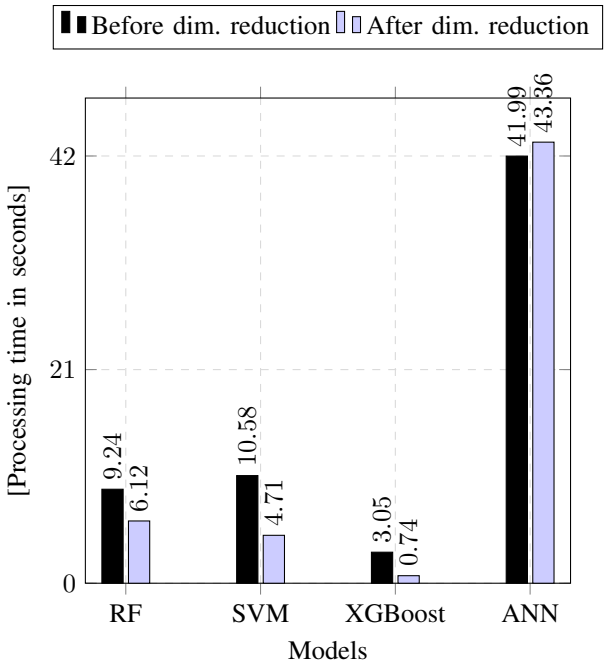


Fig. 9: Processing Time between different models

$$NECF = \frac{R_{model}}{\sum_{all} R_{model}} \tag{4}$$

$$Score = \exp\left(NECF + AC + SE + SP\right) \tag{5}$$

Table VI: Scores for each algorithm

Models	NECF	AC+SE+SP	Scores
RF	**0.5600**	2.9759	**34.327**
SVM	0.2018	2.9869	24.256
XGBoost	0.2075	2.9756	24.120
ANN	0.0307	2.9699	20.099

III. Results

We reduced 99 features into 12 by inputting the data into t-SNE model, which is illustrated in Fig. 10

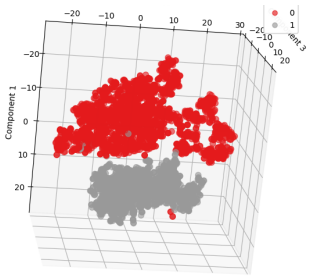


Fig. 10: Distinguishability of a high dimensional dataset with respect to its labels

It evidently shows the data set is divided into 2 categories (grey: 1, representing fall; red: 0, non-fall).

As we can see in Table VI, RF achieves the best overall performance (34.327), which is superior to other 3 algorithms. The following is SVM (24.256) and quite closed is XGBoost (24.12). ANN is the last one. The differences between them are shown in Fig. 11.

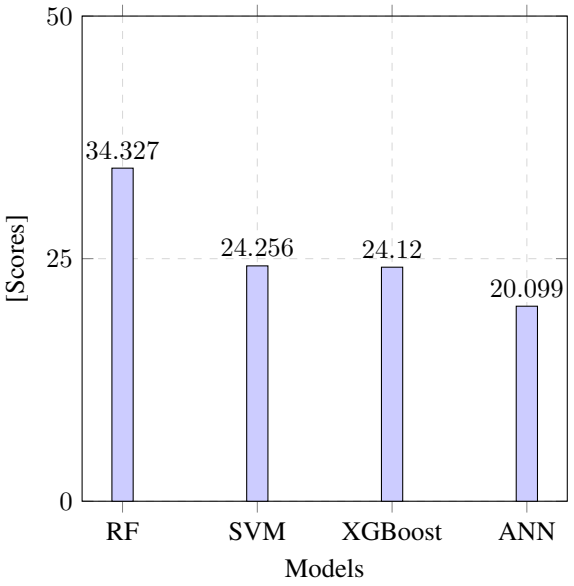


Fig. 11: Scores between different models

IV. Discussion

We estimated 3 metrics related to SM: sensitivity (SE), specificity (SP) and accuracy (AC). In addition, we also needed to consider CPU usage and processing time. Ideally, a robust algorithm should achieve high accuracy while maintaining low CPU usage and short processing time. We combined all metrics into scores. By comparison of these scores, RF (Random Forest) achieved the highest score 34.327. Due to its lowest CPU usage even with the lowest SM, it "won the game", which reveals a critical truth: we increasingly rely on deeper structured models to solve daily problems and ignore the rapid consumption of computation resources and energy. Once the development of models or architectures encounters limits, such as power restriction or maximal computation capacities, we still have to find alternative solutions to perform these tasks. In this study we designed a mechanism to measure the general performance of each algorithm, rather than focusing solely on SM as many other studies do.

XGBoost and ANN exhibited either higher CPU usage or longer processing time, placing them in the last positions. Meanwhile, SVM ranked in the middle, as its metrics were intermediate compared to the other two groups.

However, in paper [2], the authors presented an unaddressed problem which we did not solved in this study. Specifically, "after training with elderly people, the SM was still below the one obtained with young people." The dataset SisFall lacks sufficient falling cases involving elderly participants because of practical risks of implementation. Addressing this possibly requires deep machine learning models and is thus outside the main scope of this study.

V. Summary and Outlook

Health condition is one of the most extraordinary metric to measure the extent of society advance and personal development. In this study we designed a selecting method to preliminary reduction of dimensionality and mechanism to compute the scores of each algorithm during comparison. Consequently we found the most cost-effective one was RF (Random Forest).

Our next step will focus on generalization of RF algorithms. Possibly we aim to find a way to increase fall samples from elderly participants without causing physical harm. Hopefully in the next ten years, we can develop humanoids with high degrees of human likeness. At that time, we can test and collect data from the elderly-people-like robots and improve the existing economical models.

Acknowledgment

The authors would like to thank Prof. Dr. Anne Koelewijn, for her patient guidance and valuable suggestions regarding this paper.

References

[1] World Health Organization.: *Falls.*. https://www.who.int/news-room/fact-sheets/detail/falls,(2021)

[2] Angela Sucerquia.: *SisFall: A Fall and Movement Dataset.*. Sensors (Basel), 2017 Jan 20;17(1):198. doi: 10.3390/s17010198.

[3] Duan, K. et al.: *Global, regional, and national burden of falls among older adults: findings from the Global Burden of Disease Study 2021 and Projections to 2040 [J].*The International Journal for Research in Applied Science & Engineering Technology (IJRASET). 2025, (10): 1234-1245.

[4] Prathima P, et al.: *Detecting Real-Time Fall of Elderly People Using Machine Learning.*. The International Journal for Research in Applied Science & Engineering Technology (IJRASET). Volume:9 ,Issue: XII, Month of publication: December 2021

[5] Noury, et al.: *Fall detection—Principles and Methods.*. In Proceedings of the 29th Annual International Conference of the IEEE Engineering in Medicine and Biology Society, Lyon, France, 22–26 August 2007; pp. 1663–1666.